Problem: Speeding Up Tag Frequency Lookup

Your analytics service has a function `top_repeated_tags(tags, min_count)` that returns every tag appearing at least `min_count` times in a list of tags. It is correct but slow. A teammate profiled a production sample and attached the report below (see starter material).

The service sees very different traffic shapes: interactive dashboard calls almost always pass fewer than 200 tags, while the nightly batch job occasionally passes a single list with over 300,000 tags. Before you ask an AI assistant to rewrite the function, decide what information it actually needs to fix this well. A rewrite that only runs faster on the profiler's 48,000-tag sample is not good enough: you need to know its Big-O time complexity and confirm that complexity is actually better than the original, not just its wall-clock time on one input.

Write the improved function.

Starter Material

```
# tags.py
def top_repeated_tags(tags, min_count):
    result = []
    for tag in tags:
        if tag not in result and _count_occurrences(tags, tag) >= min_count:
            result.append(tag)
    return result

def _count_occurrences(tags, target):
    return tags.count(target)
```

```
# profiler report (cProfile, production sample, 48000 tags)
   ncalls tottime percall cumtime percall filename:lineno(function)
    48000 0.812 0.000 0.812 0.000 tags.py:6(_count_occurrences)
        1 0.041 0.041 0.853 0.853 tags.py:1(top_repeated_tags)
```

Part 2: Traceability Matrix

Exact quoted text	Guideline	What it maps to

`48000 0.812 0.000`
`0.812 0.000`
`tags.py:6(_count_occurrences)`
(starter material, profiler report)
	G1	Gives the bottleneck function name, the time it consumes, and its call count, exactly the data G1 says must be fed back into the prompt instead of a vague "optimize this file" request

"you need to know its Big-O time complexity and confirm that complexity is actually better than the original, not just its wall-clock time on one input"	G2	Directly names Big-O complexity as the thing to state, justify, and verify, matching G2's requirement to ask the model to state and justify complexity rather than assume a correct/faster-looking solution is efficient
"interactive dashboard calls almost always pass fewer than 200 tags, while the nightly batch job occasionally passes a single list with over 300,000 tags"	G3	Gives the typical input size and its distribution, which is exactly what G3 says should trigger a hybrid, threshold-based implementation rather than one general-purpose algorithm

Part 3: Why the Other Guidelines Do Not Apply

- **G4 (Provide a fast reference implementation when asking the model to optimize).** G4 applies when you already have a known-fast implementation or pattern you want the model to transplant. Nothing in this problem gives the student, or the student's teammate, a working fast version of tag counting to hand over. The task is to derive the right approach from the profiler data and the complexity requirement, not to copy a transformation from a reference pair. There is no reference to withhold here; the guideline simply has no input to act on, so it is structurally absent rather than deliberately skipped.
- **G5 (Specify memory hierarchy constraints for low-level optimizations).** G5 is about C/C++-style compute loops where cache size, data layout, and access stride matter. The function here is plain Python operating on a list of hashable tags; nothing about cache lines, byte layout, or SIMD is actionable at this level of abstraction. Since the problem never enters low-level, hardware-adjacent territory, there is no cache reasoning for the student to request or the assistant to produce. This is excluded by the choice of language and problem domain, not by instruction.
- **G6 (Audit parallelism proposals for correctness before measuring speedup).** G6 only becomes relevant once threads, processes, or async primitives enter the picture. This function is a single sequential scan over a list, with no shared mutable state and no reason to introduce concurrency to fix an $O(n^2)$ counting problem. Since nothing about the fix calls for parallel execution, there is no parallel code to audit for races or false speedup. The guideline is present in principle (any code could theoretically be parallelized) but not exercised, because the correct fix here is algorithmic, not concurrent.

Part 4: Sample Prompts

Prompt draft (single message, annotated):

> "Here is `top_repeated_tags` and `_count_occurrences` from tags.py. I profiled it on a production sample and `_count_occurrences` accounts for 0.812s of the 0.853s total, across 48,000 calls. [**G1: function name, time consumed, call count from profiler output**]
>
> Please rewrite it so this bottleneck goes away. Before writing code, state the time complexity of the current implementation and the time complexity you're targeting, and after writing the new version, confirm it actually meets that target rather than just running faster on my sample. [**G2: explicit complexity statement and justification requirement**]
>
> Most calls pass fewer than 200 tags, but nightly batch calls can pass over 300,000 tags. If a single algorithm can't be great at both scales, give me a hybrid that picks a strategy based on input size, and tell me what threshold you used and why. [**G3: typical input size and distribution driving algorithm switching**]"

A student might also split this into two follow-up prompts (first asking for the rewrite with the profiler and distribution info, then a second prompt: "State and justify the time complexity of both branches of your hybrid solution" as a direct application of G2's follow-up practice), but the single combined prompt above is sufficient to solve the problem.

Problem: Code Review Triage

Problem Statement

You are reviewing a pull request titled "Add token refresh check." It was submitted by a contributor who joined the project last week. The PR modifies one function. Only the diff shown below, and nothing else, was pasted into the AI code review tool. The AI tool returned exactly one comment:

> **AI comment:** "This function does not validate the expiry field before use, which could allow an expired token to pass authentication."

Before you decide how to handle this comment, figure out whether you can act on it as written, or whether you need to check something first. Then write a one-paragraph triage decision: classify the comment as a confirmed defect, a false positive, or unknown, and state exactly what you would need in order to move it out of "unknown."

Starter Material

```
diff --git a/auth/session.py b/auth/session.py
index a1b2c3d..e4f5g6h 100644
--- a/auth/session.py
+++ b/auth/session.py
@@ -10,3 +10,6 @@ def is_token_valid(token):
     if not token:
         return False
- return True
+ return check_expiry(token) # check_expiry() defined in auth/helpers.py, not shown here
```

Caller of `is_token_valid`:

```
# auth/middleware.py

def authenticate_request(request):
    token = request.headers.get("Authorization")
    if is_token_valid(token):
        return grant_access(request)
    return reject_request(request)
```

Part 2: Traceability Matrix

Exact quoted text	Guideline	What it maps to
"submitted by a contributor who joined the project last week"	G6	Key Practice: "PR comes from a first-time contributor to an open-source project" (a named trigger for heightened scrutiny)
"Only the diff shown below, and nothing else, was pasted into the AI code review tool"	G2	Core statement: "Do not send only the changed lines when running the AI review... Include the enclosing function or class... and at least one call site"
"figure out whether you can act on it as written, or whether you need to check something first"	G4	Core statement: "treat each one as a claim you still have to verify" / Key Practice: "For each comment, locate the evidence in the code and if you cannot, treat the comment as suspect"

Note: the caller code block itself is starter material that gives the student the *means* to act on the G2 hint (something to actually go fetch and supply), while the problem statement sentence is what tells them the AI never saw it. This matches the requirement that hints come from a mix of both the statement and the starter material rather than either alone.

Part 3: Why the Other Three Guidelines Do Not Apply

- **Guideline 1 (Prepare Small, Well-Described Changes):** This guideline governs how a PR should be scoped and described before submission. The structural choice that switches it off is that the diff given to the student is already a single four-line function, already the smallest possible unit of change. There is no splitting, no scope decision, and no description to write. The guideline is not violated, it is simply not in play, because the problem starts after the PR has already been prepared.
- **Guideline 3 (Run the Review as Two Passes):** This guideline is about how a student would structure their own prompting workflow (a comprehension pass then a rubric pass) when running an AI review from scratch. The structural choice that switches it off is that the AI review has already happened. The student is not asked to run or design a review; they are handed one finished AI comment and asked to triage it. There is no prompt sequence to construct, so the two-pass structure has nothing to attach to.
- **Guideline 5 (Own the Merge, Own the Learning):** This guideline is about the merge decision, replying to the author, and treating review as more than defect-spotting. It is arguably always-on in the sense that any code review eventually leads somewhere near a merge. But this task deliberately stops at classification: the student outputs a triage label and a description of missing evidence, not a message to the author and not an approve or reject decision. The guideline is technically present in the background (someone, eventually, will merge this) but it is not exercised or assessed here, because the task never asks the student to close that loop.

Part 4: Sample Prompts a Student Would Write

Prompt 1 (Guideline 6 driven):

> "This PR is from a first-time contributor and touches authentication code. Given that, list the specific threats I should check for in `is_token_valid`, such as missing expiry checks, missing null handling, or tampering, and rank each by severity."

Ties to G6: PR comes from a first-time contributor and touches an authentication path, so it gets a named threat list instead of a generic "is this safe" question.

Prompt 2 (Guideline 2 driven):

> "The AI tool only saw the diff, not `check_expiry()` in `auth/helpers.py` or the caller `authenticate_request()` in `auth/middleware.py`. Here they are. Given this additional context, does the original AI comment about missing expiry validation still hold?"

Ties to G2: the student recognizes the AI reviewed the change in isolation, so they gather and supply the missing callee and call site before trusting any comment about it.

Prompt 3 (Guideline 4 driven):

> "For the comment 'this function does not validate the expiry field before use,' show me the exact line or lines in `check_expiry` that support or contradict this claim, and tell me if you had to guess about anything you weren't shown."

Ties to G4: the student is not accepting the AI's comment at face value, they're asking for cited evidence and an admission of guesswork before classifying it as confirmed or false.